Credence Institute - AI-Driven Full Stack Software Testing Master Class/ Software Testing Syllabus

Email: Info@credence.in **Website**: https://www.credence.in

Contacts: +918237916162, +917391053250, +91 9579064658, +91 9091929355

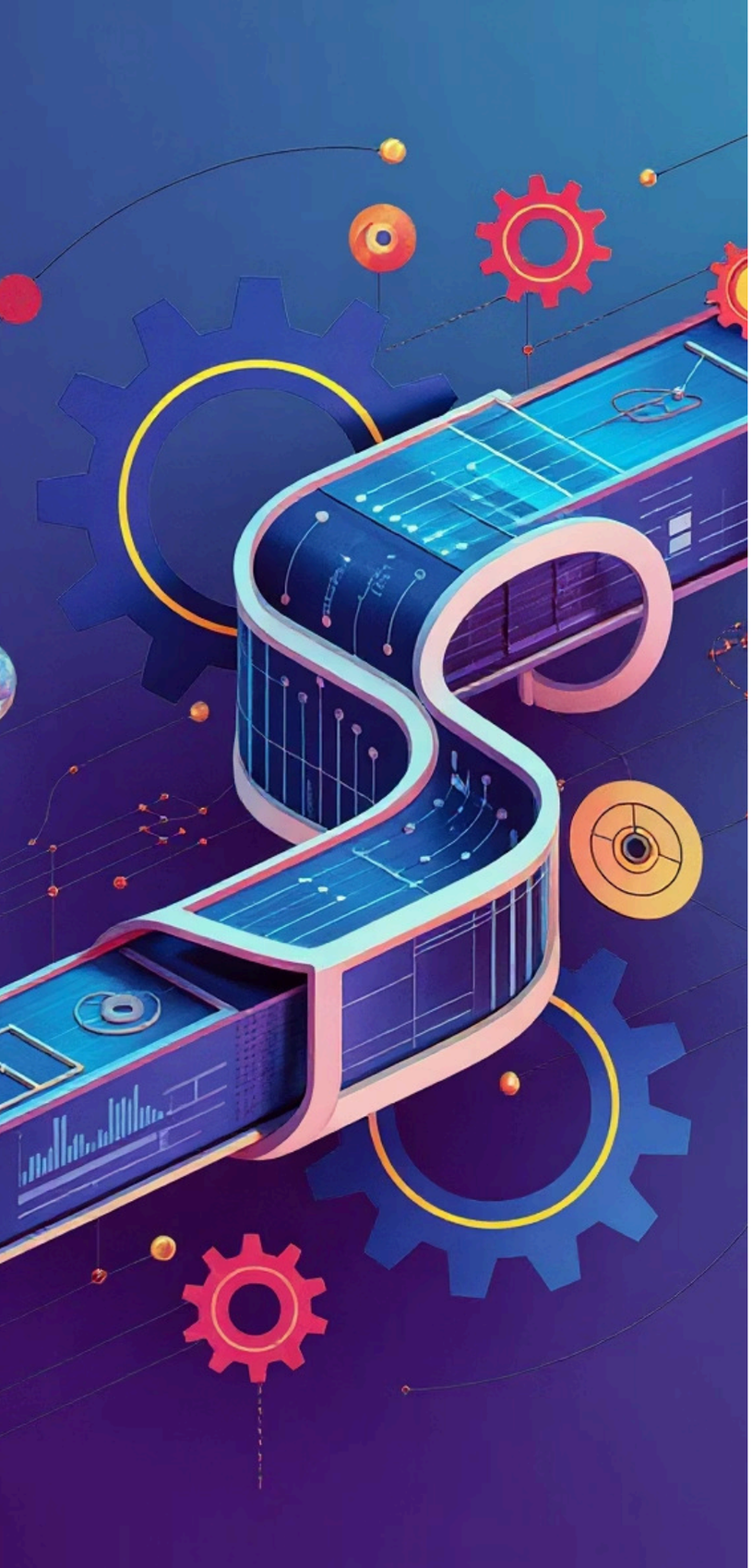
0.1 Credence Software Testing Syllabus Process

Enrollment & Training Phase

1. Enroll for batch
2. Attend Demo Session
3. Confirm the admission
4. Attend daily sessions Regularly
5. Daily practice of at least 2 hours is required

Interview Preparation & Placement

1. Attend group mock interviews
2. Attend 1-1 Telephonic interview
3. Complete the Course
4. Attend 1-1 mock interview
5. Collect the Resume
6. Attend real time Interview and clear it
7. Selection and Job is Done



02.Introduction to Software & The SDLC

Understanding Software

Software defines how computers operate. It falls into two main categories:

- **System Software:** Manages computer hardware and software resources (e.g., operating systems).
- **Application Software:** Designed to perform specific tasks for users (e.g., word processors, browsers).

We'll also touch upon bidding in the software industry and typical team structures.

Software Development Life Cycle (SDLC)

SDLC is a structured process used to develop software. It includes key phases:

- Planning & Requirement Gathering
- Design & Development
- Testing & Deployment
- Maintenance

Common SDLC models like Waterfall, V-Model, Spiral, and Agile guide these phases, each with distinct advantages.

03. Software Testing Fundamentals

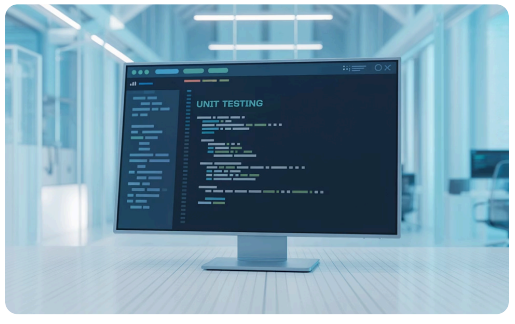
The Imperative of Testing

Software testing is a vital process that validates software functionality, performance, and security against predefined requirements. It ensures reliability and protects users and businesses from costly errors, ultimately delivering a high-quality product.

- **Why Testers?** Developers focus on creation; dedicated testers provide an objective perspective, meticulously uncovering issues that might be overlooked during development.
- **Core Concepts:** It's crucial to distinguish between an *error* (a human mistake in coding), a *defect* or *bug* (a deviation from expected behavior), and a *failure* (when the software does not meet user expectations).

04. Software Testing Levels

Software testing is systematically organized into distinct levels, each focusing on specific aspects of the system to ensure comprehensive quality assurance.



Unit Testing

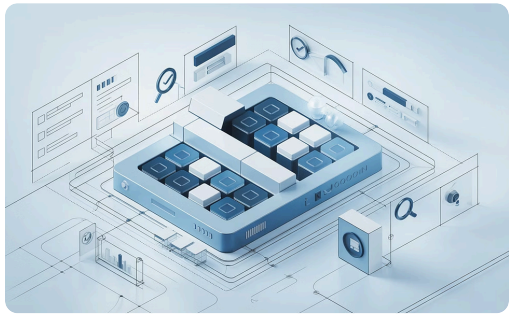
Focuses on individual components or modules in isolation to ensure they function correctly. Key techniques include:

- Basis Path Testing
- Control Structure Testing
- Conditional & Loops Coverage

Integration Testing

Verifies the interfaces and interactions between integrated software modules. Approaches include:

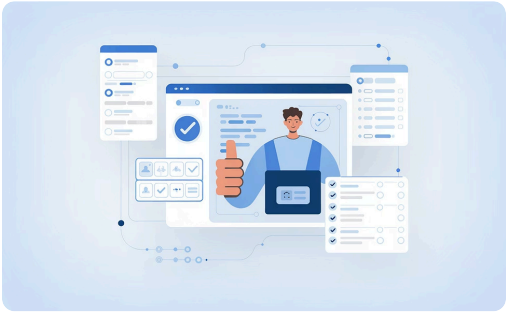
- **Incremental:** Top-Down, Bottom-Up, Sandwich
- **Non-Incremental:** Big Bang



System Testing

Evaluates the complete and integrated software system against specified requirements. It covers:

- **Usability:** GUI, Manual Support
- **Functional:** Behavioral, Input Domain, Error Handling, Backend, Service Level, Calculation
- **Non-Functional:** Recovery, Compatibility, Configuration, Inter-System, Installation, Parallel, Globalization, Sanitation, Performance, Security



User Acceptance Testing (UAT)

The final phase where end-users verify if the system meets their business needs and requirements. Types include:

- Alpha Testing (internal)
- Beta Testing (external)

05. Testing Terminology

Understanding key terminology is foundational to mastering software testing. This section defines essential concepts and processes that guide effective quality assurance.

1

Static vs. Dynamic Testing

Static Testing: Reviews code and documentation without executing the software. **Dynamic Testing:** Executes the software with test cases to observe behavior.

2

Quality Assurance (QA) vs. Quality Control (QC)

QA: Process-oriented, preventing defects. **QC:** Product-oriented, identifying defects after creation.

3

White, Black, & Grey Box Testing

White Box: Tests internal structure/code. **Black Box:** Tests functionality without knowing internal structure. **Grey Box:** Combines both, with partial knowledge of internals.

4

Software Testing Life Cycle (STLC)

A sequence of activities performed during the testing process, from requirements analysis to test closure. Ends with a formal sign-off email.

5

Environments

Distinct settings (e.g., Development, Test, Staging, Production) where software is deployed and tested, simulating real-world conditions.

6

Defect Life Cycle

The journey of a defect from discovery, reporting, fixing, retesting, to closure, involving multiple stakeholders.

7

Test Execution Life Cycle

The process of running test cases, logging results, managing defects, and generating test reports.

8

Smoke Testing & Sanity Testing

Smoke: Initial quick tests to ensure core functionality works. **Sanity:** Subset of regression tests to confirm bug fixes and new features work as intended.

9

Re-Testing & Regression Testing

Re-Testing: Verifying a specific defect fix. **Regression:** Ensuring new changes haven't negatively impacted existing functionality.

10

Exploratory Testing

Simultaneous learning, test design, and test execution, often without formal test cases, driven by the tester's experience.

11

Ad-hoc Testing

Informal, unstructured testing performed without documentation or planning, to find defects quickly.

12

Monkey Testing

Random testing where the tester (or tool) inputs arbitrary data to see if the system crashes or exhibits unexpected behavior.

13

Positive Testing

Verifies that the system behaves as expected when given valid inputs, ensuring it works according to specifications.

14

Negative Testing

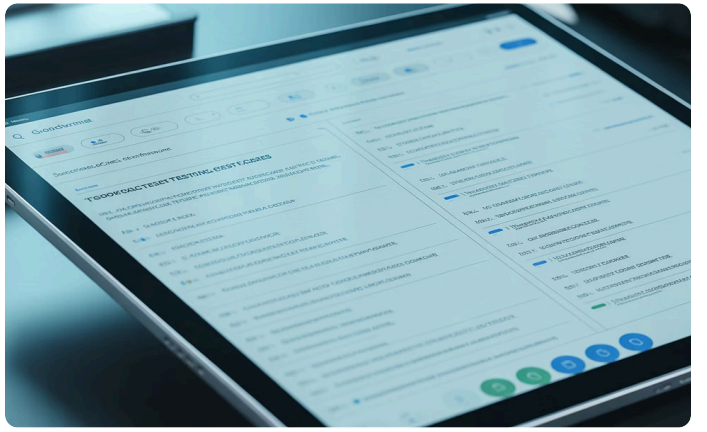
Verifies that the system handles invalid inputs gracefully, preventing crashes and displaying appropriate error messages.

06. Test Documentation Concepts

Effective software testing relies on clear, structured documentation. These key documents guide the testing process from planning to execution, ensuring thorough coverage and traceability.

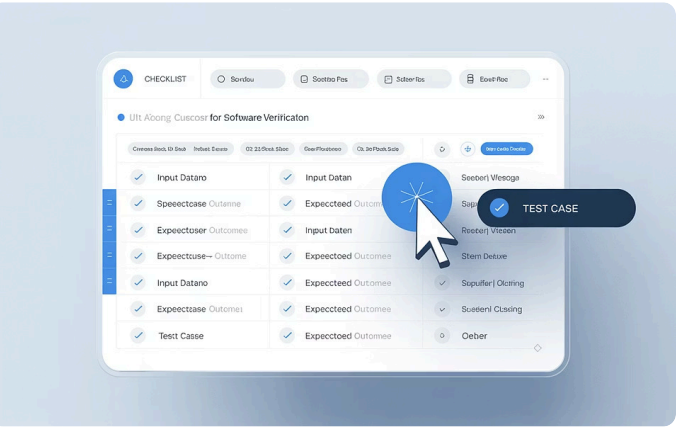
Test Plan

A high-level document outlining the scope, objectives, strategy, resources, and schedule for all testing activities. It serves as a blueprint for the entire testing project.



Test Scenarios

Descriptions of potential end-to-end user actions or system interactions that need to be tested. They address 'what to test' in broader, real-world terms, based on business requirements.



Test Case

A detailed set of steps, input data, preconditions, and expected results for verifying a specific functionality or requirement. Test cases specify 'how to test' a particular scenario.

07. Project Management Tools (HP ALM & Jira)

Mastering project management tools like HP ALM and Jira is crucial for efficient software testing. These platforms streamline various aspects of the testing life cycle:

- **Writing Test Cases:** Create and organize detailed test cases directly within the tool.
- **Release/Cycle Management:** Plan and track test cycles aligned with release schedules.
- **Executing Test Cases:** Record execution results and monitor progress.
- **Raising Defects:** Log, track, and manage bugs from discovery to resolution.
- **Severity & Priority:** Categorize defects based on impact and urgency to prioritize fixes.
- **Traceability Matrix:** Link requirements to test cases and defects, ensuring comprehensive coverage.
- **Defect Management:** Oversee the entire defect life cycle, facilitating communication and resolution.

08. SQL & Database Concepts

A strong grasp of SQL and database principles is indispensable for a software tester. This section provides the foundational knowledge and practical skills required to interact with and validate data effectively.

1

Foundational Concepts

Understand the core definitions of data, databases, and the distinction between DBMS and RDBMS, setting the stage for effective data management.

- What is Data & Database?
- DBMS vs. RDBMS

2

SQL Language Essentials

Learn the fundamental structure of SQL, including syntax, data types, and basic queries for retrieving and manipulating information.

- SQL Overview & Syntax
- Basic SQL Queries (SELECT, WHERE)
- Installation (Oracle DB, SQL Developer)

3

Advanced Operations

Dive into comprehensive SQL commands for data definition and manipulation, operators, constraints, functions, and powerful join types.

- DDL, DML, DCL, TCL Commands
- Operators, Constraints & Functions
- SQL Joins (Inner, Outer, Full)

4

Querying & Design

Master complex data retrieval using clauses, subqueries, and explore database normalization for optimal structure and integrity.

- SQL Clauses (GROUP BY, HAVING, ORDER BY)
- Subqueries & Aggregation
- Normalization, Case Statements

10. Advanced SQL Concepts

Deepen your understanding of SQL with powerful features that enhance data manipulation, query performance, and database programmability. Mastering these advanced concepts is crucial for complex data analysis and robust application development.

Programmable Objects & Structures

- Store Procedures: Reusable blocks of SQL code.
- Triggers: Automated actions in response to database events.
- Views: Virtual tables based on query results.
- Materialized Views: Pre-computed and stored views for performance.
- Temporary Tables: Short-lived tables for intermediate results.
- Indexing: Optimizing data retrieval speed.

Advanced Functions & Expressions

- COALESCE & NVL: Handling NULL values.
- TRIM / LTRIM / RTRIM: Cleaning string data.
- ROUND & CAST: Numeric and data type conversions.
- Character, Date, Conversion Functions: Specialized data manipulation.
- REPLACE: Substituting strings within text.
- Regression Expressions: Pattern matching with regular expressions.

Analytical & Optimization Techniques

- PARTITION BY & GROUP BY: Grouping and aggregating data.
- WITH Clause (CTEs): Common Table Expressions for complex queries.
- Pivoting: Transforming rows into columns for reporting.
- Query Optimization Concepts: Improving query efficiency.
- Window Functions (ROW_NUMBER(), RANK(), etc.): Advanced analytical functions.
- Recursive Queries: Processing hierarchical data.

11. Database Types in Syllabus

In this section, you'll gain practical experience with leading database management systems essential for modern software testing. Understanding these platforms is crucial for validating data integrity, querying complex datasets, and ensuring robust application performance.

Oracle Database

Explore the world's most popular enterprise database. You'll learn to interact with Oracle for data retrieval, manipulation, and schema understanding, vital for large-scale application testing.

Microsoft SQL Server

Master SQL Server, a robust relational database widely used in corporate environments. Focus on its specific T-SQL syntax, tools, and best practices for effective data validation and performance testing.

Azure-Cloud Database

Delve into cloud-based database solutions with Azure. Understand how cloud databases differ from on-premises systems, focusing on scalability, availability, and specific testing considerations for cloud-native applications.

12. Database Testing

Database testing is a critical phase in the software development lifecycle, ensuring the accuracy, integrity, and performance of data stored within the application's backend. It validates that all database operations—from data manipulation to schema integrity—function as intended, preventing data loss, corruption, and system failures.

Why Database Testing?

Database testing confirms data integrity, validates business rules, checks data mapping, and verifies performance under various loads. It's essential for preventing data-related bugs that can lead to significant operational issues and data inconsistencies.

Benefits of Database Testing

- Ensures data accuracy and reliability.
- Prevents data corruption and loss.
- Improves system performance and scalability.
- Enhances application stability.
- Reduces maintenance costs by catching issues early.

Key Areas of Database Testing

→ Schema Testing

Verifies that the database schema (tables, columns, indexes, views, stored procedures, triggers, etc.) is correctly defined and aligns with application requirements.

→ SQL Queries Testing

Validates the correctness and efficiency of SQL queries used by the application, ensuring accurate data retrieval and manipulation.

→ Data Integrity Testing

Confirms the accuracy, consistency, and validity of data throughout its lifecycle, including data types, constraints, and relationships between tables.

→ Functional Testing

Ensures that database operations triggered by application functionality (e.g., CRUD operations) behave as expected according to business rules.

13. Azure Cloud Concepts

This section introduces the fundamentals of cloud computing and dives into key Microsoft Azure services, providing essential knowledge for testing cloud-native applications and data solutions.

What is Cloud Computing?

Understand the on-demand delivery of IT resources and applications over the internet, covering its benefits and operational models.

Types of Cloud Services

- **IaaS**: Infrastructure as a Service
- **PaaS**: Platform as a Service
- **SaaS**: Software as a Service

Introduction to Microsoft Azure

Explore Azure's comprehensive suite of cloud services, understanding its global infrastructure and core capabilities for modern enterprises.

Key Azure Services for Testing

Azure Data Bricks

- Notebooks & SQL queries
- Cluster management
- Data loading & permissions

Azure Data Factory

- Pipeline creation & validation
- Server checking & monitoring

Azure Synapse Analytics

- SQL query & Data Frames
- Comprehensive query validation

Azure DevOps

- Sprint & story creation
- Bug tracking & linking

14. Performance Testing with JMeter

Performance testing evaluates an application's responsiveness, stability, and scalability under varying workloads. It's crucial for identifying bottlenecks and ensuring optimal user experience before deployment.

Stress Testing

Evaluates system behavior beyond normal operating conditions to find its breaking point and error handling capabilities.

Load Testing

Verifies system performance under expected and peak user loads to ensure it meets service level agreements.

Key JMeter Concepts

Test Plan & Threads

Define the test scenario (Test Plan) and simulate multiple virtual users (Threads) interacting with the application.

HTTP(s) Requests

Simulate user actions by sending various HTTP and HTTPS requests to web servers or APIs.

Assertions & Listeners

Validate server responses (Assertions) and visualize performance metrics (Listeners) in real-time or post-execution.

We'll also explore BlazeMeter for generating JMeter scripts, setting up automation test scenarios, executing tests in non-GUI mode, and creating comprehensive performance dashboards.

15. API Testing using Postman

API (Application Programming Interface) testing is crucial for ensuring the reliability, security, and performance of the backbone of modern applications. In this module, you will master Postman, a leading tool for designing, developing, and testing APIs, enabling you to validate direct communications between different software components.

Overview of Postman

Gain a comprehensive understanding of Postman's interface, features, and capabilities as the industry-standard tool for API development and testing.

Key API Request Types

GET Request

Retrieve data from a specified resource without altering it, typically for fetching information.

POST Request

Send data to a server to create a new resource, such as submitting form data.

PUT Request

Update an existing resource on the server with new data, replacing the entire resource.

DELETE Request

Remove a specified resource from the server, permanently erasing its data.

Login Functionality Testing

Test various authentication methods for secure access to APIs, ensuring proper authorization and data protection.

No Authentication

Test public APIs or endpoints that do not require any form of authentication.

Basic Authentication

Implement and test username/password-based authentication for protected resources.

Bearer Token

Work with token-based authentication (e.g., OAuth 2.0, JWT) for secure API calls.

Integration Testing with Postman

Learn to chain multiple API requests together, simulating real-world user flows and verifying the end-to-end functionality of interconnected services within a system.

16. Salesforce CRM

Salesforce CRM is the world's leading cloud-based platform for customer relationship management, helping companies connect with their customers in a whole new way and streamline operations.

What is CRM?

CRM, or Customer Relationship Management, is a technology for managing all your company's relationships and interactions with customers and potential customers.

What is Salesforce?

Salesforce is a comprehensive, cloud-based software company that provides CRM services and a suite of enterprise applications focused on customer service, marketing automation, and analytics.

What is Cloud?

In the context of Salesforce, "Cloud" refers to delivering computing services—including servers, storage, databases, networking, software, analytics, and intelligence—over the Internet.

17. Data Warehousing – ETL + BI Testing

This section delves into the foundational concepts of data warehousing, Extract, Transform, Load (ETL) processes, and Business Intelligence (BI) testing, crucial for validating robust data solutions.

What is a Data Warehouse?

A centralized repository of integrated data from one or more disparate sources, used for reporting and data analysis.

Understanding ETL

Explore the Extract, Transform, Load process—a critical step in data integration that moves data from sources to the data warehouse.

Introduction to Data Marts

Learn about data marts as subsets of a data warehouse, catering to specific business functions or departments for targeted analysis.

OLTP vs. OLAP

Differentiate between Online Transaction Processing (OLTP) for daily operations and Online Analytical Processing (OLAP) for complex analysis.

Data Warehouse Architecture & Flow

Understand the high-level design (HLD) and technical flow of data within a data warehouse, from source systems to end-user reporting tools, including data ingestion, transformation, and storage.

18. ETL and Report Testing, Tools and SQL Server/Oracle IDE

This module explores the critical domain of Extract, Transform, Load (ETL) and Report Testing, ensuring data integrity and accuracy across complex data workflows. We will cover essential tools and methodologies for robust data validation.

Key ETL & Database Tools

SQL Server Integration Services (SSIS)

Master the design and deployment of data integration and workflow applications for efficient ETL processes.

Informatica

Gain proficiency in a leading enterprise data integration platform for managing and transforming large datasets.

SQL Server Management Studio (SSMS)

Utilize this integrated environment for managing SQL Server infrastructure, including database development and administration.

ETL Testing Fundamentals

- **What is ETL Testing?**

  Understand the process of validating data after extraction, transformation, and loading into a data warehouse, ensuring data quality and consistency.

- **ETL vs. Database Testing**

  Learn the key distinctions between testing the ETL process itself (data movement and transformation) versus testing the database for schema and data integrity.

- **Testing Scenarios & Test Cases**

  Develop comprehensive test scenarios and specific test cases for various ETL stages, covering data completeness, transformation accuracy, and performance.

- **Data Mapping Documents**

  Analyze and utilize "God's Documents" or Data Leakage Documents to verify the accurate mapping and flow of data from source to target.

Practical ETL Project & Data Validation

We will apply these concepts through the **EMP_ETL Project**, focusing on:

01

Project Scenarios

Define and execute test scenarios for a simulated Employee ETL project, covering various data conditions and business rules.

02

Comparing Data

Write and execute SQL queries to compare source and target data, identifying discrepancies and ensuring accurate transformations.

03

Data Types

Work with both live and test data to simulate real-world ETL operations and validate data loading types (initial, incremental, full load).

ETL and Report Testing Challenges

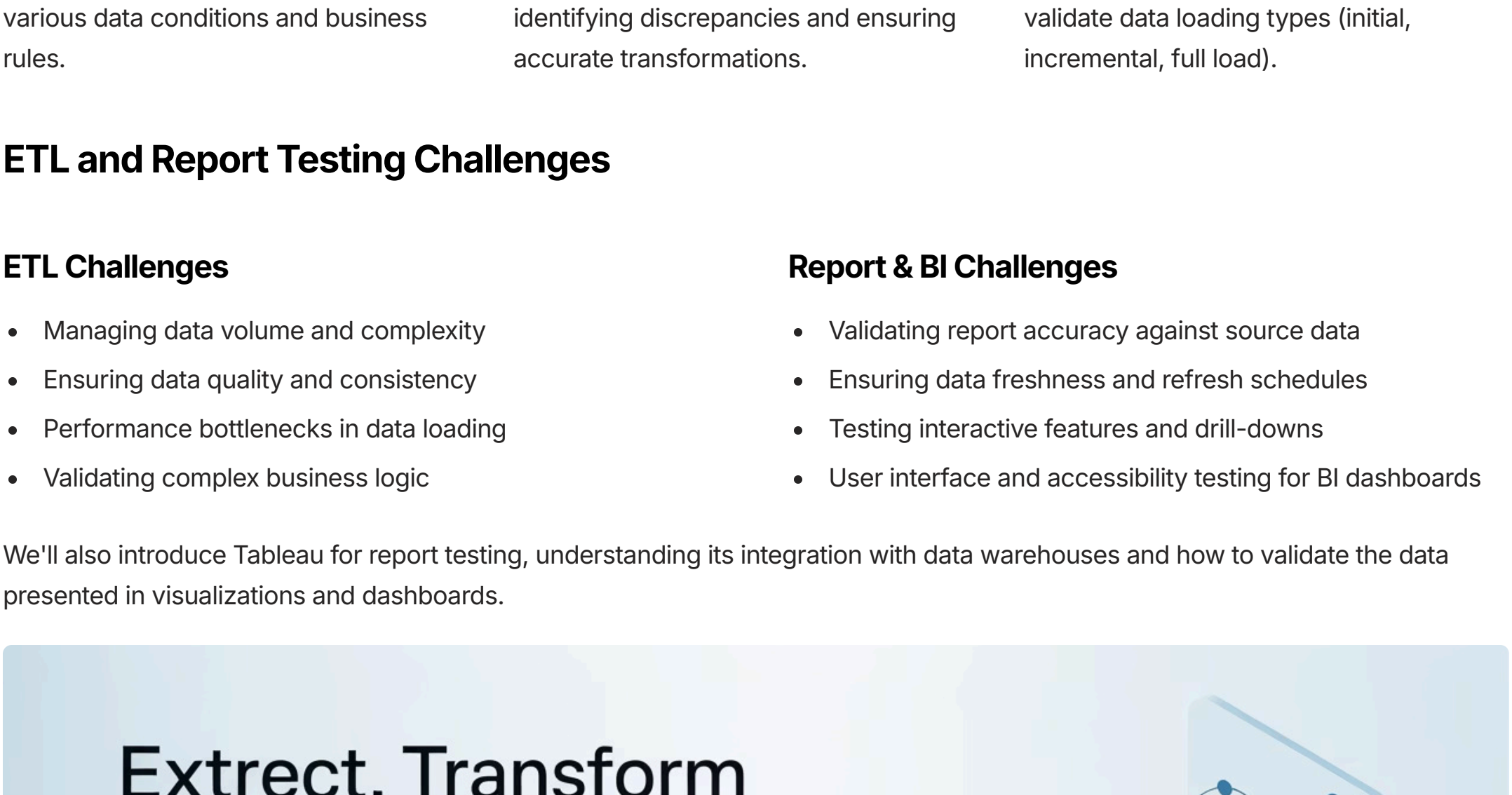
ETL Challenges

- Managing data volume and complexity
- Ensuring data quality and consistency
- Performance bottlenecks in data loading
- Validating complex business logic

Report & BI Challenges

- Validating report accuracy against source data
- Ensuring data freshness and refresh schedules
- Testing interactive features and drill-downs
- User interface and accessibility testing for BI dashboards

We'll also introduce Tableau for report testing, understanding its integration with data warehouses and how to validate the data presented in visualizations and dashboards.



19. Power BI for Report Testing

This module focuses on Power BI, Microsoft's leading business intelligence tool, and its critical role in data visualization and report testing within modern data warehousing ecosystems.

Key Aspects of Power BI and Report Testing

Power BI Overview

Explore Power BI's capabilities as a business intelligence tool for interactive data visualization and comprehensive reporting.

Need for Power BI

Understand its significance in data-driven decision-making, consolidating diverse data sources, and creating accessible, insightful reports.

Functional & Integration Testing

Learn to validate data accuracy, calculated measures, and the seamless integration of Power BI reports with underlying data sources.

Report & KPI Validation

Verify visual integrity, filter functionality, drill-down capabilities, and the precise representation of Key Performance Indicators (KPIs).

Scenario for Power BI Testing

We'll walk through a practical scenario to apply functional and integration testing principles, ensuring the reliability of Power BI dashboards and reports.

20. Introduction to Automation Testing with Python

This module provides a comprehensive introduction to Python programming, focusing on the fundamental concepts essential for developing robust automation testing scripts.

Basic Syntax & Variables

Learn Python's foundational syntax, including defining variables, understanding naming conventions, and utilizing various operators for calculations and comparisons.

Core Data Types

Explore Python's built-in data types: numbers, strings, booleans, lists, tuples, sets, and dictionaries, mastering their creation and manipulation for data handling.

Control Flow & Functions

Master conditional statements (if/elif/else), looping constructs (for/while), and function creation, including lambda functions, to build dynamic program logic.

File Handling & Exception Handling

Understand how to read from and write to files, manage file methods, and implement robust error handling using try-except blocks for reliable scripts.

Object-Oriented Programming (OOP)

Gain insights into OOP principles with Python, covering classes, objects, constructors, methods, inheritance, encapsulation, and polymorphism for structured and reusable code.

21. Selenium Components

Dive into Selenium, the industry-standard framework for automating web browser interactions. This module equips you with the essential tools and techniques to build robust and efficient web application tests using Python.

Setup & Core Components

Selenium Installation

Learn to install the Selenium library using `pip`, integrating it with your Python development environment.

Python IDE Setup

Configure your preferred Python Integrated Development Environment (IDE) for optimal Selenium test script development and management.

Web Driver Configuration

Understand how to set up and manage specific browser drivers for Chrome, Firefox, and Edge to enable browser automation.

Selenium WebDriver Fundamentals

Browser Invocation

- Chrome
- Firefox
- Edge

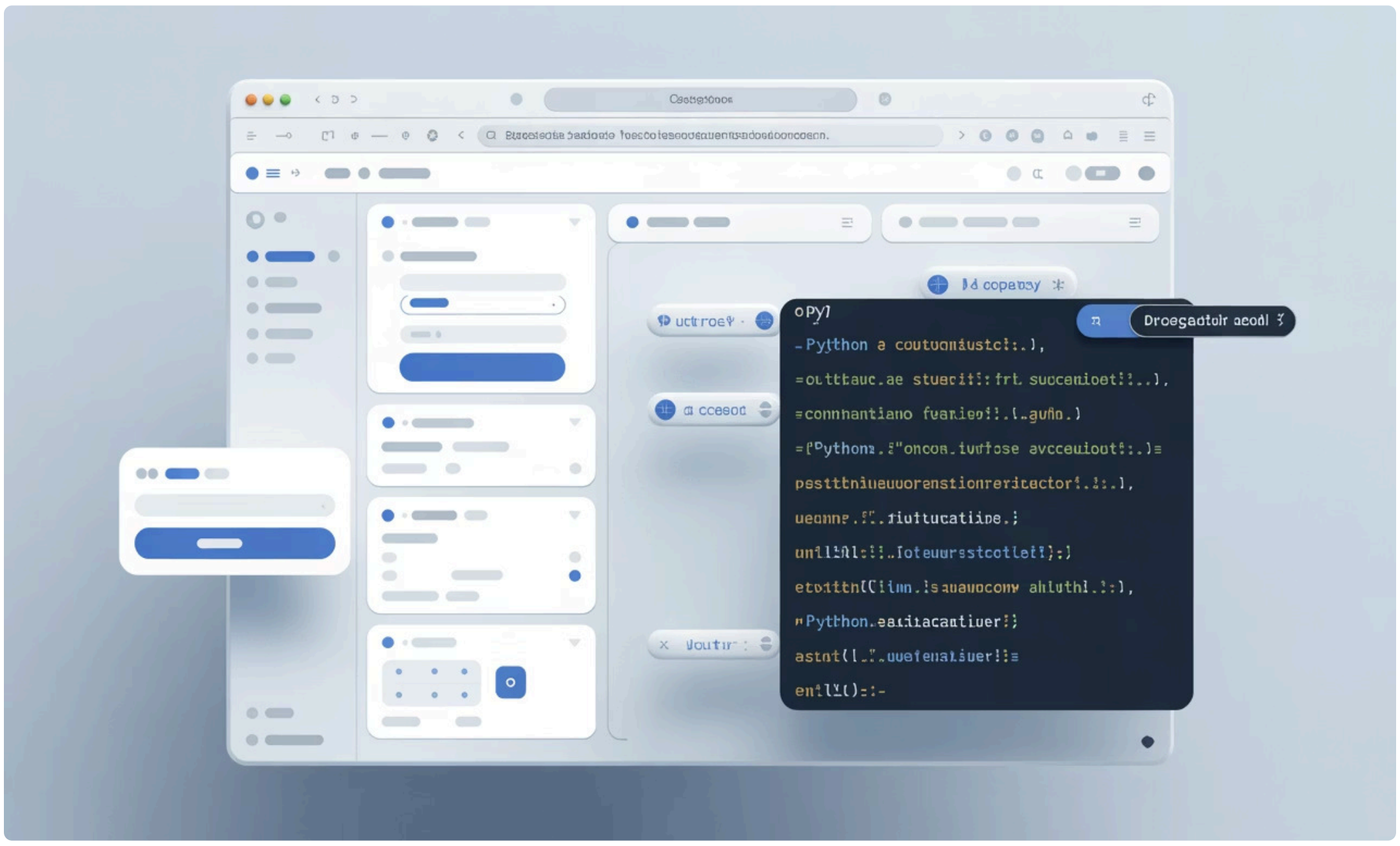
Locators

Master various strategies to precisely identify web elements for interaction:

- ID, Name, Class Name, Tag Name
- Link Text, Partial Link Text
- CSS Selector
- XPath

Working With Elements

- **Interacting with UI Elements**
  - Handling Alerts, Modals, and Pop-ups
  - Managing Calendars and Time Pickers
  - Selecting Checkboxes and Radio Buttons
  - Performing Mouse Actions: Click & Hold, Double Click, Right Click, Mouse Hover
  - Implementing Drag & Drop Functionality
  - Interacting with Dropdown Menus
  - Uploading Files
- **Advanced Interactions & Waits**
  - Checking Element Display Status
  - Implementing Explicit and Implicit Waits for Synchronization
  - Working with iFrames
  - Switching between Browser Tabs and Windows
  - Automating Web Tables, including those with Fixed Headers



22. Automation Testing Framework (PyTest)

This module delves into the world of automation testing frameworks, focusing specifically on PyTest – a powerful and flexible framework for Python.

What is a Testing Framework?

A testing framework is a set of guidelines, tools, and best practices designed to streamline the testing process. It provides a structured approach to writing, organizing, and executing tests, ensuring efficiency, maintainability, and reusability of test code.

Introducing PyTest Framework

PyTest is a popular, open-source Python testing framework known for its simplicity and extensibility. It facilitates writing concise and readable test cases, scaling from simple unit tests to complex functional testing for APIs and web applications.

Optimized Test Execution

Explore features like **parallel test runs** for faster execution, **markers** for categorizing and selecting tests, and built-in support for generating informative **HTML reports**.

CI/CD Integration

Learn to integrate PyTest with continuous integration/continuous deployment (CI/CD) pipelines using tools like **Jenkins** for automated builds and **GitHub** for version control and collaborative development.

Test Structure & Configuration

Master effective **test case writing** practices and utilize the `conftest.py` **file** for fixture management, plugin loading, and sharing configurations across multiple test files.

Advanced Test Data Handling

Implement **data-driven testing** using **parameterization** for varied test inputs and leverage **Excel** for managing larger datasets (DDT – Data-Driven Testing).

Comprehensive Reporting & Logging

Understand how to implement robust **log generation** for debugging and auditing, and integrate with **Allure Reports** for highly interactive and detailed test reports.

We'll apply these concepts through hands-on projects, building a PyTest framework from scratch, implementing Page Object Model, configuring for multiple browsers, and integrating all the advanced features.

23. Unix Fundamentals

This module introduces the foundational concepts of Unix and Linux operating systems, essential for understanding server environments and advanced testing methodologies. You'll gain practical experience with fundamental commands for file management, text processing, and system interaction.

Unix vs. Linux: Understanding the Differences

Unix

- Proprietary, commercial operating system.
- Developed in the 1960s at Bell Labs.
- Less flexible, but highly stable and secure.
- Used mainly in servers, mainframes, and workstations.

Linux

- Open-source, free operating system kernel.
- Developed by Linus Torvalds in 1991.
- Highly customizable and diverse distributions.
- Used in servers, desktops, mobile devices, and embedded systems.

Key Unix Commands & Concepts

File System Architecture

Explore the hierarchical structure of Unix/Linux file systems, including root directory, home directories, and common system folders like `/bin`, `/etc`, and `/var`.

File Management Basics

Learn to create (`touch`), open (`cat`, `less`, `more`), and copy (`cp`) files and directories, mastering fundamental interactions with the file system.

Content Inspection

Utilize `head` and `tail` commands to view the beginning or end of files, and `grep` for powerful pattern searching within text content.

Advanced Text Processing

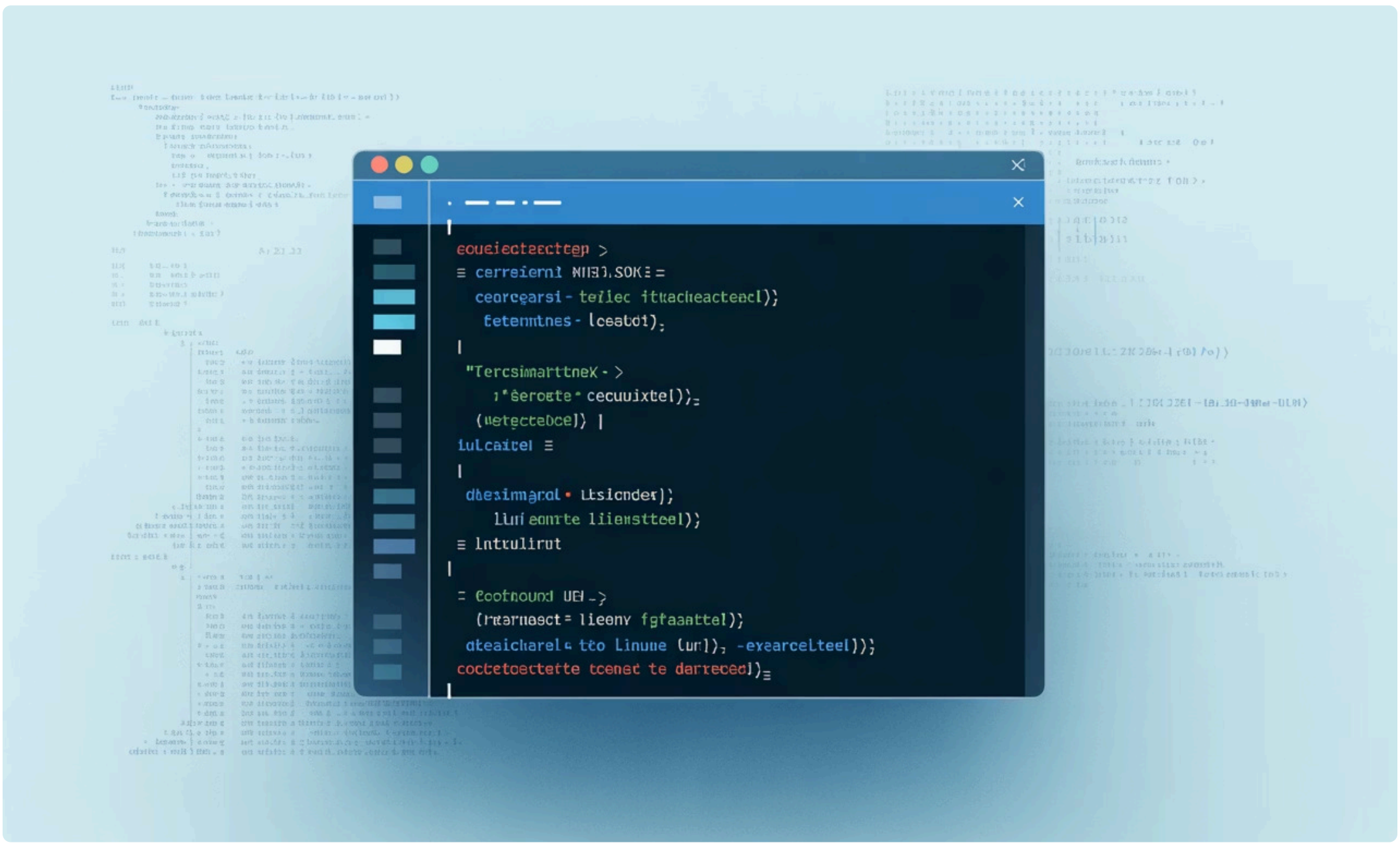
Discover the power of `sed` (stream editor) for sophisticated text transformations and command-line editing, crucial for data manipulation.

File Permissions (chmod)

Understand and apply `chmod` to manage file and directory permissions, controlling read, write, and execute access for users and groups.

Finding Files

Master commands like `find` and `locate` to efficiently search for files and directories based on various criteria across the file system.



24. Key Tools and Software for Testing

Mastering modern software testing requires proficiency with a diverse set of tools. This section provides an overview of the essential software and platforms covered in our masterclass, empowering you to navigate complex testing environments.

Database & Data Tools

- SQL Developer
- SQL Server Management Studio
- Visual Studio + SSIS

BI & Reporting

- Tableau
- Power BI

Project & Test Management

- HP ALM
- Jira
- Git
- Jenkins

Automation & Performance

- PyCharm
- Postman
- JMeter

Application & Cloud

- Salesforce CRM
- Azure Databricks

Utilities & AI Assistants

- Unix Putty
- Notepad++
- Excel
- Outlook
- ChatGPT
- DeepSeek
- Grok AI

This comprehensive toolkit ensures you're prepared for any challenge in the software testing landscape, from data analysis to advanced automation and AI-driven insights.

25. Project/Domain Training

Understanding the specific business domain is crucial for effective software testing. This module provides focused training on various industry sectors, equipping you with the contextual knowledge needed to excel in specialized testing roles. We'll delve into the unique challenges, regulatory requirements, and key functionalities of different industries.

Telecom & Billing

Explore complex systems for communication, billing, operations support (OSS), and mediation, focusing on service delivery and revenue assurance.

Capital Markets

Dive into investment banking platforms, trading systems, portfolio management, and regulatory compliance within the financial sector.

Healthcare Domain

Learn about electronic health records (EHR), patient management systems, medical devices, and adherence to stringent healthcare regulations like HIPAA.

Card Payment Systems

Focus on the security, functionality, and compliance of credit/debit card processing, payment gateways, and fraud detection systems.

Life Sciences

Understand systems used in pharmaceuticals, biotechnology, and medical research, including clinical trials, drug discovery, and regulatory submissions.

Insurance Domain

Examine software for policy administration, claims processing, actuarial calculations, and customer relationship management in the insurance industry.

Automotive Domain

Gain insights into embedded systems, infotainment, autonomous driving features, and manufacturing processes within the automotive sector.

For each batch, we select 3-4 specific domains to ensure in-depth coverage and practical application, allowing you to develop specialized expertise relevant to current industry demands.

Thank You!

We appreciate your engagement and look forward to seeing you succeed in your software testing career. For any further questions, please don't hesitate to reach out.

CREDENCE IT PROFESSIONAL

TRAINING INSTITUTE, PUNE

Email: Info@credence.in **Website**: https://www.credence.in

Contacts: +918237916162, +917391053250, +91 9579064658, +91 9091929355